

SnapGestão — Resumo Completo do Projeto

Gerado em: 08/05/2026 | Versão do app: 1.0.8 (build 9) | Branch: master → main | Autor: Diego Manhães

1. Visão Geral

SnapGestão é um aplicativo mobile de controle financeiro pessoal desenvolvido com React Native + Expo 54. O foco é em *potes de orçamento* mensais por ciclo customizável, com funcionalidades avançadas de IA, metas, reserva de emergência, OCR de cupons fiscais e análise de preços colaborativa.

Item	Valor
Plataforma	Android (foco), iOS planejado
Framework	React Native · Expo SDK 54 · Expo Router ~6.0.23
Linguagem	TypeScript (strict)
Backend	Supabase (Postgres + RLS + Edge Functions + Storage)
Estado	Zustand (working set) + React Query (fetch/cache)
Offline	expo-sqlite — estrutura pronta, sync não implementado
Supabase URL	https://cvyissbkfwphtmvmvcvop.supabase.co
GitHub Pages	https://snapgestao-cpu.github.io/snapgestao/

Repositório	github.com/snapgestao-cpu/snapgestao
-------------	--------------------------------------

2. Stack e Dependências Críticas

Lib	Versão	Observação
expo	~54	Nunca atualizar sem atualizar expo-router junto
expo-router	~6.0.23	PINADO — incompatível com versões maiores no SDK 54
@supabase/supabase-js	^2	Auth + DB + Storage
zustand	^4	stores/: useAuthStore, useCycleStore, usePotStore
@tanstack/react-query	^5	hooks/: usePots, useTransactions, useIncomeSources
expo-sqlite/legacy	—	Offline local DB (snapgestao.db)
expo-secure-store	—	LargeSecureStoreAdapter com memoryCache em lib/supabase.ts
expo-print + expo-sharing	—	Geração e compartilhamento de PDF
expo-media-library	—	Salvar PDF em Downloads (nunca documentDirectory)
@react-native-community/datetimepicker	—	Seleção de datas nativa Android

New Architecture	habilitada	newArchEnabled: true — evitar libs incompatíveis
------------------	------------	--

Regra de instalação: sempre usar `npm install <pkg> --legacy-peer-deps`

3. Estrutura de Arquivos

Pasta / Arquivo	Responsabilidade
<code>app/(auth)/</code>	Login, register
<code>app/(tabs)/</code>	Tabs: monthly, index (potes), projection, goals, profile
<code>app/(tabs)/_layout.tsx</code>	Tab bar com badge de lançamentos pendentes
<code>app/_layout.tsx</code>	Route guard principal — auth → termos → onboarding → tabs
<code>app/onboarding/</code>	Wizard 3 steps: saldo → ciclo/fontes → primeiro pote
<code>app/pot/[id].tsx</code>	Detalhe do pote, lançamentos, agendados
<code>app/ocr.tsx</code>	Leitura de cupom fiscal: QR→SEFAZ + OCR fallback
<code>app/mentor.tsx</code>	Mentor Financeiro IA (quiz + análise + PDF)
<code>app/analizador-precos.tsx</code>	Analizador de Preços IA (quiz + comparativo)
<code>app/achievements.tsx</code>	Grid de badges/conquistas
<code>app/terms.tsx</code>	Aceite LGPD — bloqueante até aceite

<code>lib/</code>	Toda lógica de negócio (sem UI)
<code>stores/</code>	Zustand: <code>useAuthStore</code> , <code>useCycleStore</code> , <code>usePotStore</code>
<code>hooks/</code>	React Query hooks
<code>components/</code>	Componentes reutilizáveis
<code>constants/colors.ts</code>	Tokens de cor — sempre usar daqui, nunca hardcoded
<code>types/index.ts</code>	User, Pot, Transaction, Goal, CreditCard, IncomeSource
<code>supabase/migrations/</code>	SQL migrations para rodar manualmente no Supabase
<code>supabase/functions/</code>	Edge Functions Deno deployadas
<code>docs/</code>	GitHub Pages: index, termos, privacidade, email-confirmado
<code>assets/</code>	Imagens: carteira.png (logo), potes/, icon_v1.png

4. Arquitetura e Fluxo de Dados

4.1 Route Guard (app/_layout.tsx)

Ordem de verificação a cada mudança de estado de auth:

1. Loading → `ActivityIndicator`
2. Não autenticado → `/(auth)/login`
3. Autenticado + `terms_accepted_at` IS NULL ou versão desatualizada → `/terms` (`gestureEnabled: false`)
4. Autenticado + sem onboarding → `/onboarding/step1`
5. OK → `/(tabs)/monthly` (tela inicial)

Regra crítica: quando `isAuthenticated=true` mas `user=null` (perfil ainda carregando), o guard aguarda — nunca

redireciona prematuramente. Nunca adicionar `getSession()` extra em `_layout.tsx`.

4.2 Fluxo de Dados

React Query (fetch/cache) → supabase.* → onSuccess → Zustand store Componentes leem do Zustand (síncrono, sem suspense)

Nunca chamar `supabase` diretamente de componentes.
Exceções documentadas: `useAuthStore`,
`onboarding/step3.tsx`, `app/(tabs)/index.tsx`.

4.3 Ciclos

- **Crédito:** filtra por `billing_date`
- **Tudo mais:** filtra por `date`
- Sincronização entre Potes e Mensal via `useCycleStore.cycleOffset` (range -24 a +12)
- `monthly.tsx` usa `computeCycleSummaryFromData` (síncrono) — nunca `calculateCycleSummary`

4.4 SecureStore / Auth Performance

`LargeSecureStoreAdapter` em `lib/supabase.ts` tem:

- `memoryCache` — evita re-leituras de disco
- `pendingReads` — deduplicação de leituras paralelas da mesma chave
- Reduz `getSession` de ~8s para <1s
- `clearSecureStoreCache()` exportado para uso no `logout/delete-account`

5. Banco de Dados

5.1 Tabelas Principais

Tabela	Descrição
users	Perfil + cycle_start + initial_balance + onboarding_completed + terms_accepted_at + terms_version

income_sources	Fontes de receita fixas por usuário
income_source_history	Histórico de valores por fonte × mês (valid_from)
pots	Potes de orçamento — soft delete via deleted_at
pot_history	Histórico de nome/limite por pote × mês (valid_from)
pot_limit_history	Histórico de mudanças de limite com valid_from por ciclo
transactions	Lançamentos financeiros — installment_group_id para parcelamento
credit_cards	Cartões de crédito do usuário
cycle_rollovers	Saldo de encerramento de ciclo (surplus_action, surplus_goal_id)
goals	Metas financeiras — status: active/completed/cancelled
goal_transactions	Movimentações por meta (deposit_external, deposit_from_cycle, withdrawal_to_cycle)
emergency_reserve	Reserva de emergência (1 row por usuário)
emergency_reserve_transactions	Histórico de movimentações da reserva
scheduled_transactions	Lançamentos agendados (orçados) — start_date, total_months
scheduled_transaction_months	1 row por mês de cada agendado — status: pending/confirmed/cancelled

price_database	Base colaborativa de preços de cupons NFC-e (30 dias de retenção)
user_preferences	share_price_data BOOLEAN NULL
user_badges	Badges/conquistas ganhos por usuário
smart_merchants	Estabelecimentos reconhecidos automaticamente
receipts	Cupons fiscais processados via OCR

projection_entries — REMOVIDA. Dropar se existir: `DROP TABLE IF EXISTS public.projection_entries;`

5.2 Regras Críticas de Potes

Nunca usar `.is('deleted_at', null)` sozinho — perde pots soft-deleted para mês futuro.
 Nunca ler `name/limit_amount` direto de pots para views passadas — usar `lib/pot-history.ts`.
 Nunca incluir `icon` ou `mesada_active` em `INSERT/UPDATE` de pots — colunas não existem.

5.3 payment_method válidos

`cash · debit · credit · pix · transfer · voucher_alimentacao · voucher_refeicao`

Nunca usar `'other'` — não é válido no DB. Fallback: `'cash'`.

5.4 Migrations (rodar em ordem no Supabase SQL Editor)

1. `20240418_cycle_rollovers.sql`
2. `20240419_pot_soft_delete_and_history.sql`
3. `20240420_pots_physical_delete.sql` — FK `transactions.pot_id` → `ON DELETE SET NULL`
4. `20240422_onboarding_completed.sql`
5. `20240501_scheduled_transactions.sql`
6. `20240503_income_source_history.sql`
7. `20240504_emergency_reserve.sql`
8. `20240505_goal_transactions.sql`
9. `20240506_terms.sql`

20240421_pots_display_order.sql — NÃO aplicar (feature revertida).

6. Features Implementadas

6.1 Auth e Onboarding

- Login/register via Supabase Auth
- Wizard 3 steps: saldo inicial (opcional, pode ser zero) → ciclo + fontes de renda → primeiro pote
- Nunca usar `initial_balance === 0` como guard — saldo zero é válido
- Trigger `on_auth_user_created` ativo no Supabase

6.2 Termos de Uso / LGPD

- Tela `/terms` bloqueante (`gestureEnabled: false`) — mostra antes do onboarding
- Dois checkboxes independentes (Termos + Política de Privacidade)
- Links para GitHub Pages: `termos.html` e `privacidade.html`
- Versão atual dos termos: `'1.0'` em `TERMS_VERSION`
- Perfil mostra data do aceite e versão

6.3 Potes (Dashboard)

- Grid de potes com JarPot (visualização PNG por percentual de uso)
- Histórico de estado por mês via `lib/pot-history.ts`
- Soft delete — pote permanece visível em ciclos passados
- Lançamentos agendados (badge no tab Potes) via `lib/scheduled-transactions.ts`
- `NewPotModal`: create, edit, reativar, duplicate prevention

6.4 Mensal

- Navegação por ciclo com offset (−24 a +12)
- Summary: receita base + extra + rollover anterior − despesas = saldo
- Dois alerts: vermelho (déficit) e âmbar (pote excedeu limite)
- "Encerrar ciclo" — cascadeia `recalculateRollover`
- Importação de arquivo Excel/CSV

- Tela inicial após login

6.5 Projeção

- 13 rows (meses passados + atual + futuros)
- Meses passados: até 3 consecutivos com dados reais
- Futuros: totalBudgeted (limites de pote) + excedente de parcelas + lançamentos reais
- Otimizado com `getPotsHistoryBatch` (2 queries para todos os offsets)

6.6 Metas

- Metas de longo prazo com simulação de juros compostos
- Depósito externo, depósito do ciclo, saque para ciclo
- Histórico de movimentações por meta
- Status: active / completed / cancelled
- Metas concluídas em modal separado

6.7 Reserva de Emergência

- 1 row por usuário (`emergency_reserve`)
- Depósito externo, do ciclo e saque para ciclo
- Meta de valor opcional
- Card no topo da tela de Metas

6.8 Histórico de Fontes de Receita

- Alterações valem do mês escolhido em diante (não retroativas)
- `getIncomeSourcesForMonth` e `getIncomeSourcesBatch` para projeção otimizada

6.9 Lançamentos Agendados

- Orçamento mensal recorrente por pote
- Confirmar → cria transaction real + marca status: confirmed
- Badge no tab Potes via `useCycleStore.pendingScheduledCount`

6.10 OCR / NFC-e

- Path 1 (recomendado): QR Code → SEFAZ (RJ, SP, MG suportados)

- Path 2 (fallback): foto → Edge Function `process-receipt` → Google Vision
- `sanitizeNFCEurl()` chamado uma vez no `ocr.tsx`, nunca dentro de `NFCEWebView`
- `NFCEWebView`: polling interno (15×1s) para SEFAZ-RJ com jQuery Mobile

6.11 IA (Mentor + Analisador)

- Provider: Claude Haiku (`claude-haiku-4-5-20251001`), Gemini 2.5 Flash, Llama 3.3 70B (Groq)
- Provider padrão: `'claude'` — persiste em `useCycleStore.aiProvider`
- Groq — descrição: "Groq — Leve e eficiente" (sem badge "GRÁTIS")
- Mentor: quiz 5 perguntas + análise + PDF salvo em Downloads
- Analisador: quiz 3 perguntas + comparativo de preços por estabelecimento
- Limite de tokens por usuário — ver `supabase/scripts/grant_ai_tokens.sql`

6.12 Base de Preços Colaborativa

- Coleta anônima de itens de cupons NFC-e (opt-in por usuário)
- Sem `user_id` — nunca inclui dados pessoais
- Limpeza automática após 30 dias via Edge Function `cleanup-price-database`
- Toggle "Compartilhar preços anônimos" no Perfil

6.13 Gamification

- 10 badges definidos em `lib/badges.ts`
- `BadgeToast` com fila de animação (slide-in + fadeOut 3s)
- Cooldown de 1h no startup para não reprocessar a cada abertura
- Tela `app/achievements.tsx` com grid de badges

6.14 Exportação

- Excel: 1 aba por mês + aba Resumo; 5 presets de período
- IR CSV: lançamentos do ano anterior
- PDF: Mentor e Analisador via `lib/gerar-pdf.ts` — salvo em Downloads (MediaLibrary)

6.15 Exclusão de Conta

- Edge Function `delete-account` deployada em `cvyissbkfwphtmvvcvop`
- Deleta dados de 13 tabelas + remove `auth.users` via service role
- Botão "⚠ Zona de Perigo" no Perfil com dupla confirmação
- Após exclusão: limpa SecureStore cache → `signOut` → redirect para login

7. Edge Functions (Supabase Deno)

Função	Status	Descrição
<code>process-receipt</code>	☑ Ativa	OCR via Google Cloud Vision — extrai merchant, total, data, CNPJ, itens. Salva em bucket receipts.
<code>delete-account</code>	☑ Ativa	Deleta todos os dados do usuário + <code>auth.users</code> . Exige JWT válido.
<code>cleanup-price-database</code>	☑ Ativa	Remove registros de <code>price_database</code> com mais de 30 dias.
<code>fetch-nfce</code>	🏰 Tombstoned (HTTP 410)	SEFAZ-RJ bloqueia IPs de datacenter. Parsing agora é client-side.

8. Regras e Decisões Críticas

8.1 Regras que nunca devem ser violadas

Regra	Motivo
-------	--------

Potes sempre via <code>lib/pot-history.ts</code>	Sem histórico = estado incorreto em meses passados
Crédito filtra por <code>billing_date</code> , outros por <code>date</code>	Ciclo financeiro correto
<code>monthly.tsx</code> usa <code>computeCycleSummaryFromData</code>	Síncrono, zero queries extras
<code>_layout.tsx</code> nunca adiciona <code>getSession()</code> extra	<code>init()</code> já cuida — duplicar causa race condition
Onboarding guard: nunca <code>initial_balance === 0</code>	Saldo zero é válido — causaria loop infinito
<code>payment_method</code> nunca <code>'other'</code>	Não é válido no DB — usa constraint CHECK
PDF salvo via <code>MediaLibrary + Downloads</code>	<code>documentDirectory</code> não aparece para o usuário
<code>sanitizeNFCeUrl()</code> só em <code>ocr.tsx</code>	NFCeWebView recebe URL já sanitizada
Sem imports de <code>expo-notifications</code>	Notificações completamente desabilitadas
Groq sem badge GRÁTIS	Decisão de produto — descrição: "Groq — Leve e eficiente"

8.2 Performance — não regredir

- Sem N+1 em queries de pote — usar `enrichWithHistory` com `.in()`
- Emergency pot balance sempre dentro de `Promise.all`
- Projeção usa `getPotsHistoryBatch` (2 queries para todos os offsets)
- `monthly.tsx` transaction queries com `.limit(200)`
- `ImportFileModal` montado condicionalmente em `monthly.tsx`
- `Single NewPotModal` com `key={editingPot?.id ?? 'new'}`

8.3 Bugs Android — não reintroduzir

- **Button bounce:** nunca KeyboardAvoidingView no nível da tela — só dentro de Modal
- **Modal layout corruption:** bottom sheet e overlay como siblings, não parent-child
- **storage.removeItem is not a function:** já corrigido via ExpoSecureStoreAdapter em lib/supabase.ts

9. GitHub Pages (docs/)

Arquivo	URL
docs/index.html	https://snapgestao-cpu.github.io/snapgestao/
docs/termos.html	https://snapgestao-cpu.github.io/snapgestao/termos.html
docs/privacidade.html	https://snapgestao-cpu.github.io/snapgestao/privacidade.html
docs/email-confirmado.html	https://snapgestao-cpu.github.io/snapgestao/email-confirmado.html

Logo carteira.png copiada para docs/ e usada por caminho relativo em todos os HTMLs. Header com gradiente azul (#0F5EA8 → #1a7fd4), logo 96×96px, "Controle Financeiro Pessoal" em branco (color: white explícito em header p).

10. Variáveis de Ambiente (.env — nunca commitar)

```
EXPO_PUBLIC_SUPABASE_URL=https://cvyissbkfwphtmvvcvop.supabase.co EXPO_PUBLIC_SUPABASE_ANON_KEY=... EXPO_PUBLIC_GEMINI_API_KEY=... EXPO_PUBLIC_ANTHROPIC_API_KEY=... EXPO_PUBLIC_GROQ_API_KEY=...
```

EXPO_PUBLIC_* são inlined pelo Metro em build time. Secrets de backend nunca devem usar este prefixo.

11. Comandos Principais

Comando	Uso
<code>npm start</code>	Metro Bundler (dev)
<code>npm run android</code>	Rodar no Android (emulador ou dispositivo)
<code>npx tsc --noEmit</code>	Type-check — rodar antes de qualquer commit importante
<code>npm install <pkg> --legacy-peer-deps</code>	Instalar dependências
<code>npm run build:android</code>	APK release (via build.ps1)
<code>npm run build:android:debug</code>	APK debug
<code>npm run prebuild</code>	Regenerar android/ ios/ — DESTRUTIVO
<code>npx supabase functions deploy <nome> --project-ref cvyissbkfwphmtmvvcvop</code>	Deploy de Edge Function
<code>git push origin master:main</code>	Push padrão (branch local: master → remoto: main)

Build de Release (build.ps1)

1. Commit assets + app.json e push
2. Para processos Gradle
3. Deleta pasta android/
4. `npx expo prebuild --clean --no-install`
5. `.\gradlew assembleRelease`
6. Copia APK como SnapGestao-v{version}-build{versionCode}.apk

Versão atual: **1.0.8 (build 9)** — incrementar em `app.json` antes do próximo build.

12. Arquivos Mortos (podem ser deletados)

- `components/ProjectionEntryModal.tsx` — não importado em nenhum lugar
- `components/charts/BarChart.tsx` — não importado em nenhum lugar

13. Próximos Passos (Roadmap)

Prioridade	Tarefa	Observações
Alta	Build de produção (EAS)	Incrementar versão para 1.0.9 / build 10 em app.json antes de rodar build.ps1
Alta	Testes e validações finais	Testar fluxo completo: onboarding → termos → potes → metas → exclusão de conta
Média	Push notifications	Requer build de produção (expo-notifications desabilitado em dev)
Média	Glossário financeiro	Modal educativo com termos usados no app
Baixa	Sync offline	expo-sqlite pronto, falta implementar sync bidirecional com Supabase
Baixa	Suporte iOS	Testar e ajustar layouts para iOS
Baixa	Deletar arquivos mortos	ProjectionEntryModal.tsx e charts/BarChart.tsx

Para o próximo build de release:

1. Atualizar `app.json`: `version` → "1.0.9" e `versionCode` → 10
2. Rodar `.\build.ps1` no PowerShell (demora ~15 min)
3. APK gerado em
`C:\snapgestao\snapgestao\SnapGestao-v1.0.9-build10.apk`

14. Histórico de Commits Recentes

Hash	Mensagem
d53c4a7	feat: exclusão total de conta via Edge Function delete-account
3382360	fix: logo maior e texto branco nos headers dos documentos legais
8fa5c9d	fix: logo carteira.png na tela de aceite de termos LGPD
6d05171	fix: logo carteira.png nos documentos legais do GitHub Pages
c3600b7	fix: URLs dos documentos legais apontando para GitHub Pages
3a2dcfd	feat: páginas legais em HTML para GitHub Pages, página de email confirmado
f2fce83	feat: tela de aceite LGPD, Termos de Uso e Política de Privacidade
76e27cb	docs: cabeçalhos e comentários em todos os arquivos
0b3a929	build: atualizar ícones e versao
7d67c18	feat: mostrar anos e meses no badge da meta, botão excluir meta